

Eksamens-huskelapp — SQL og ER-diagram (krakefot)

Komprimert referanse for eksamen. Eksempler i SQLite-/standard-SQL-stil.

Logisk utføringsrekkefølge i SELECT (ikke skriverekkefølgen):

1. **FROM / JOIN** — bygg råtabellen
2. **WHERE** — filtrer rader
3. **GROUP BY** — grupper rader
4. **HAVING** — filtrer grupper
5. **SELECT** — velg/regn ut kolonner (alias laget her)
6. **DISTINCT** — fjern duplikater
7. **ORDER BY** — sortér
8. **LIMIT / OFFSET** — begrensn antall rader

Konsekvens: alias laget i SELECT kan brukes i ORDER BY, men ikke i WHERE/GROUP BY/HAVING (med mindre motoren tillater det).

1. Grunnleggende setningsstruktur

```
SELECT [DISTINCT] kolonner | uttrykk [AS alias]
FROM tabell [AS alias] [JOIN ... ON ...]
WHERE radbetingelser
GROUP BY kolonner
HAVING gruppebetingelser
ORDER BY kolonner [ASC|DESC]
LIMIT n [OFFSET m];
```

2. WHERE — filtrering av rader

Operator	Eksempel	Merknad
= <> < <= > >=	pris > 100	Standard sammenligning
AND / OR / NOT	aar=2025 AND land='NO'	Bruk parentes når du blander AND/OR
BETWEEN x AND y	aar BETWEEN 1990 AND 1999	Inklusive begge ender
IN (...)	land IN ('NO','SE')	Kortere enn flere OR
NOT IN (...)	id NOT IN (SELECT ...)	Pass på NULL i sub-spørring
LIKE '%a_'	navn LIKE 'P%'	% = 0+ tegn, _ = ett tegn
IS NULL / IS NOT NULL	kommentar IS NULL	NULL kan IKKE testes med =
EXISTS / NOT EXISTS	EXISTS (SELECT 1 FROM ...)	NULL-trygg variant av IN

3. JOIN-typer

INNER JOIN

Bare rader som matcher i *begge* tabeller.

```
SELECT b.tittel, f.navn
FROM bok b
JOIN forfatter f ON b.fid = f.id;
```

LEFT (OUTER) JOIN

Alle rader fra venstre tabell. Manglende høyre → NULL.

```
SELECT k.navn, b.id
FROM kunde k
LEFT JOIN bestilling b ON b.kid = k.id;
```

RIGHT / FULL JOIN

Speilet av LEFT, hhv. begge sider med NULL der match mangler. Mange motorer (SQLite) støtter ikke disse — bytt rekkefølge og bruk LEFT JOIN.

SELF JOIN

Tabell joinet med seg selv (med ulike alias). Brukt for hierarkier (sjef/ansatt).

```
SELECT a.navn, s.navn AS sjef
FROM   ansatt a
JOIN   ansatt s ON a.sjef_id = s.id;
```

CROSS JOIN

Kartesisk produkt — alle kombinasjoner. FROM a, b uten ON gir det samme.

Anti-join (finn det som mangler)

```
SELECT k.* FROM kunde k
LEFT JOIN bestilling b ON b.kid = k.id
WHERE  b.id IS NULL;
```

Kunder uten bestilling.

4. Aggregat og GROUP BY

Funksjon	Hva den gjør
COUNT(*)	Teller alle rader (også NULL).
COUNT(kol)	Teller rader der kol ikke er NULL.
COUNT(DISTINCT kol)	Teller unike ikke-null-verdier.
SUM, AVG	Summerer / snitt over ikke-null-verdier.
MIN, MAX	Minste / største verdi.

```
SELECT land, COUNT(*) AS antall, AVG(pris) AS snitt
FROM   film
WHERE  aar >= 2000
GROUP BY land
HAVING COUNT(*) > 5
ORDER BY antall DESC;
```

Regel: Alle ikke-aggregerte kolonner i SELECT må også stå i GROUP BY (i streng SQL).
WHERE filtrerer før aggregering, **HAVING** filtrerer etter.

5. Sub-spørringer (subquery)

Skalar (returnerer 1 verdi)

```
SELECT * FROM film
WHERE pris > (SELECT AVG(pris) FROM film);
```

IN / NOT IN (liste)

```
SELECT * FROM kunde
WHERE id IN (SELECT kid FROM bestilling);
```

Korrelert (refererer til ytre rad)

```
SELECT a.* FROM ansatt a
WHERE lønn = (
  SELECT MAX(lønn)
  FROM   ansatt
  WHERE avd = a.avd);
```

Høyest lønn i hver avdeling.

Inline view (FROM-subspørring)

```
SELECT avd, snitt FROM (
  SELECT avd, AVG(lønn) AS snitt
  FROM   ansatt GROUP BY avd) t
WHERE snitt > 50000;
```

6. NULL — tre-verdi-logikk

- `NULL = NULL` → *UKJENT* (ikke sant). Bruk `IS NULL`.
- `NULL + 1`, `NULL || 'x'` → `NULL`.
- Aggregater (`SUM`/`AVG`/`MIN`/`MAX`/`COUNT(kol)`) ignorerer `NULL`. `COUNT(*)` teller alt.
- `COALESCE(a, b, c)` → første ikke-`NULL`.
- `NOT IN (...)` blir *tomt resultat* hvis subspørringen kan returnere `NULL` — bruk `NOT EXISTS`.

7. Sortering, paginering og strenger

```
SELECT navn, pris FROM film
ORDER BY pris DESC, navn ASC
LIMIT 10 OFFSET 20;      -- topp 10, hopp over 20
```

Funksjon	Eksempel
Strengkonkat	<code>fornavn ' ' etternavn</code> (SQLite/PG); <code>CONCAT(...)</code> (MySQL)
<code>UPPER / LOWER</code>	<code>UPPER(navn) = 'OLA'</code>
<code>SUBSTR(s, start[, len])</code>	<code>SUBSTR(navn, 1, 3)</code>
<code>LENGTH(s)</code>	Antall tegn
<code>ROUND(x [,n])</code>	<code>ROUND(snitt, 2)</code>

8. Datofunksjoner (SQLite)

```
SELECT DATE('now'), DATETIME('now', '-7 days');
SELECT strftime('%Y', dato) AS aar FROM bestilling;
WHERE dato >= DATE('now', 'start of month');
```

`%Y`=år, `%m`=mnd, `%d`=dag, `%w`=ukedag (0=søndag).

9. CASE – betinget uttrykk

```
SELECT navn,  
CASE  
  WHEN pris < 100 THEN 'Billig'  
  WHEN pris < 500 THEN 'Middels'  
  ELSE 'Dyr'  
END AS kategori  
FROM film;
```

10. UNION / INTERSECT / EXCEPT

- UNION stabler to spørringer, fjerner duplikater. UNION ALL beholder dem (raskere).
- Antall kolonner og datatyper må stemme.
- INTERSECT = felles rader. EXCEPT = i A, men ikke i B.

11. DML – endre data

```
INSERT INTO kunde (navn, land) VALUES ('Ola', 'NO');  
INSERT INTO kunde (navn) SELECT navn FROM gammel_kunde;  
  
UPDATE film SET pris = pris * 1.1 WHERE aar < 2000;  
  
DELETE FROM kunde WHERE id = 7;
```

Glem aldri WHERE i UPDATE/DELETE — uten WHERE rammes hele tabellen.

12. DDL – opprette tabeller

```
CREATE TABLE kunde (  
  id          INTEGER PRIMARY KEY,  
  navn        TEXT NOT NULL,  
  epost        TEXT UNIQUE,  
  registrert  DATE DEFAULT (DATE('now'))  
);  
  
CREATE TABLE bestilling (  
  id          INTEGER PRIMARY KEY,  
  kid         INTEGER NOT NULL,  
  belop       REAL CHECK(belop >= 0),  
  FOREIGN KEY(kid) REFERENCES kunde(id)  
);
```

13. Vanlige feller på eksamen

- Glemmer parentes rundt OR i WHERE: WHERE a=1 AND (b=2 OR c=3).
- Bruker = mot NULL i stedet for IS NULL.
- Glemmer kolonner i GROUP BY (de som ikke er aggregert).
- Bruker WHERE i stedet for HAVING på et aggregat: WHERE COUNT(*)>1 → feil.
- LIMIT uten ORDER BY — rekkefølge er udefinert.
- «Mangler» rader i resultatet etter join? Sjekk om det burde vært LEFT JOIN.
- NOT IN med subspørring som returnerer NULL → tomt sett.
- Forskjell mellom COUNT(*) og COUNT(kolonne) når kolonnen kan være NULL.

ER-diagram — Krakefot-notasjon (Crow's Foot)

Også kalt Chen/Martin-notasjon med krakefot. Brukes til å vise tabeller (entiteter) og hvordan de henger sammen (relasjoner) i en database.

14. Symbolene på linjen

Endene av linjen mellom to entiteter forteller *hvor mange* det kan være, og om det er *obligatorisk*. Hver ende har **to symboler**: indre symbol = minimum, ytre symbol = maksimum.

Symbol	Betyr	Eksempel-tekst
	Nøyaktig én (1..1, obligatorisk)	«Hver bestilling har én og bare én kunde»
0	Null eller én (0..1, valgfri)	«En ansatt kan ha én partner (eller ingen)»
<	Én eller flere (1..N, obligatorisk mange)	«En faktura har minst én linje»
0 <	Null eller flere (0..N, valgfri mange)	«En kunde kan ha 0..N bestillinger»

Hvordan lese symbolene: Symbolet nær *den andre* entiteten forteller hvor mange av *denne* entiteten som hører til én av *den andre*. Det er den vanlige fellen — les alltid «*hvor mange B per A*» ved siden av A.

15. Kardinaliteter — de tre hovedtilfellene

1 : 1 (én-til-én)

PERSON ||—|| PASS

Hver person har ett pass; hvert pass tilhører én person.

Mapping: slå sammen til én tabell, eller legg FK på den ene siden med **UNIQUE**.

1 : N (én-til-mange)

KUNDE ||—0< BESTILLING

Én kunde kan ha 0..N bestillinger; én bestilling har akkurat én kunde.

Mapping: FK på *mange-siden* (BESTILLING.kunde_id → KUNDE.id).

M : N (mange-til-mange)

STUDENT >0—0< FAG

En student kan ha mange fag; et fag har mange studenter.

Mapping: krever *koblingstabell* med to FK-er som primærnøkkel sammen.

16. Eksempel: les diagrammet

KUNDE ||—0< BESTILLING ||—|< ORDRELINJE >|—|| PRODUKT

- **KUNDE — BESTILLING:** 1 kunde har 0..N bestillinger; hver bestilling har 1 kunde.
- **BESTILLING — ORDRELINJE:** 1 bestilling har 1..N linjer (obligatorisk minst én); hver linje hører til 1 bestilling.
- **ORDRELINJE — PRODUKT:** 1..N linjer per produkt; hver linje peker på 1 produkt.

17. Fra ER til relasjoner (mapping-regler)

ER-konstruksjon	Bli til i tabellene
Entitet (rektangel)	Egen tabell. Primærnøkkel = identifikator-attributt.
Attributt	Kolonne i tabellen. <i>Sammensatt</i> attributt → flere kolonner. <i>Flerverdi</i> → egen tabell.
1 : N relasjon	Fremmednøkkel på <i>mange-siden</i> .
1 : 1 relasjon	FK på én av sidene + UNIQUE ; eller slå sammen til én tabell.
M : N relasjon	Egen koblingstabell med (FK_a, FK_b) som sammensatt PK. Eventuelle relasjons-attributter legges her.
Svak entitet	Tabell med PK = (FK til eier, eget delvis ID). Total deltakelse mot eier.
Total deltakelse (heltrukken/« » innerst)	NOT NULL på FK.
Valgfri deltakelse («0» innerst)	FK kan være NULL.

18. Eksempel: M:N med koblingstabell

```
STUDENT(sid, navn)
FAG(fkode, tittel)
TAR(sid, fkode, semester)      -- koblingstabell
  FOREIGN KEY (sid) REFERENCES STUDENT(sid)
  FOREIGN KEY (fkode) REFERENCES FAG(fkode)
```

Sammensatt primærnøkkel (sid, fkode) hindrer at samme student tar samme fag dobbelt. Relasjons-attributtet `semester` hører hjemme i koblingstabellen.

19. Sjekkliste for å tegne et ER-diagram

1. Finn entitetene (substantivene): KUNDE, ORDRE, PRODUKT...
 2. Sett opp attributter; marker primærnøkkel (understrek).
 3. Finn relasjonene (verbene): «kunde *legger inn* ordre».
 4. For hver relasjon, spør to ganger:
 - *Hvor mange B kan én A ha?* → ytre symbol nær B.
 - *Må A ha minst én B?* → indre symbol nær B.
- Gjenta motsatt retning.
5. Identifiser M:N — lag koblingstabell med eventuelle relasjons-attributter.
 6. Sjekk svake entiteter (eksisterer bare gjennom en eier).
 7. Skriv ut tabellene: PK understreket, FK med pil/notasjon.

Huskeregel for FK-plassering (1:N): «FK-en bor på mange-siden». Hvis du er i tvil — den siden som *peker mot 1* er der FK-en hører hjemme.

20. Mini-ordliste (norsk)

Norsk	Engelsk / SQL
Entitet	Entity (≈ tabell)
Attributt	Attribute (≈ kolonne)
Primærnøkkel	Primary key (PK)
Fremmednøkkel	Foreign key (FK)
Kardinalitet	Cardinality (1, N, M)
Deltakelse	Participation (total / partial)
Koblingstabell	Junction / linking table
Svak entitet	Weak entity
Krakefot	Crow's foot

— Lykke til på eksamen —